

OCA I3C Controller — Low-Level Design

OCA I3C Controller — Low-Level Design (Internal)

1. Module structure
2. Per-instance state (`inst_state`)
3. Processes
 - 3.1 Transaction scheduling
4. CCI parameter catalogue
5. Register decode
 - 5.1 Threshold / interrupt level
6. Transaction engine (`process_command`)
7. IBI injection (`inject_ibi`)
8. Output driving (`output_method`)
9. Single-driver & determinism
10. Build / packaging

Style: SystemC / TLM-2.0 Loosely-Timed (LT) · **CCI:** 1.0

1. Module structure

The CCI parameters are declared **before** the ports so they are constructed first; `num_instances_p.get_value()` then sizes every output `sc_vector` (matching the `reset_unit` ordering discipline).

Each instance carries:

- `pio_intr_status` holds only the **latched** (W1C) bits; the level threshold bits are computed on demand from queue occupancy (§5).

3. Processes

Process	Kind	Sensitivity	Purpose
b_transport	TLM cb	—	Decode instance + offset, dispatch reg read/write.
transport_dbg	TLM cb	—	Back-door DAT/DCT access (no side effects).
output_method	SC_METHOD	recompute_event_	Sole driver of all output signals.
xfer_method	SC_METHOD	xfer_event_	Process one queued command per xfer_delay_.

output_method is the only process that writes the output signals (single driver rule). All mutation paths (reg_write, FIFO-port reads, inject_ibi, process_command) call schedule_recompute() which notifies recompute_event_ at SC_ZERO_TIME.

3.1 Transaction scheduling

enqueue_command() (and HC_CONTROL enabling the bus) pushes the instance index onto xfer_pending_ and notifies xfer_event_ at xfer_delay_. xfer_method pops one instance, processes a single command, and re-notifies while xfer_pending_ is non-empty — a self-perpetuating drain that serialises transactions at one per xfer_delay_.

4. CCI parameter catalogue

Name	Type	Default	Mutability	Metadata
num_instances	unsigned	6	immutable	rtl_param=NUM_I3C, valid_range=1..6
access_delay_ns	double	2.0	mutable	unit=nanoseconds
xfer_delay_ns	double	100.0	mutable	unit=nanoseconds
cmd_fifo_depth	unsigned	8	immutable	command/response queue
rx_fifo_depth	unsigned	64	immutable	RX DWORDs
tx_fifo_depth	unsigned	64	immutable	TX DWORDs
ibi_fifo_depth	unsigned	8	immutable	IBI DWORDs

The constructor validates ranges and raises SC_REPORT_FATAL on num_instances $\notin [1,6]$, negative delays, or a zero FIFO depth. Resolved values are mirrored into cfg_ and used everywhere downstream.

5. Register decode

`b_transport` computes `inst = addr / 0x500`, `loff = addr % 0x500`, then dispatches to `reg_read(inst, loff, data)` / `reg_write(inst, loff, data)`.

- **DAT/DCT windows** (`0x300..0x3FF`, `0x400..0x4FF`) index directly into the `dat` / `dct` arrays.
- **FIFO ports** have read/write side effects:
 - `COMMAND_PORT` write: two-write 64-bit assembly → `enqueue_command`.
 - `XFER_DATA_PORT` write → TX push; read → RX pop.
 - `RESPONSE_PORT` read → response pop.
 - `IBI_PORT` read → IBI pop.
- **Computed read-only** registers: `HC_CAPABILITIES`, `PRESENT_STATE`, `DAT_SECTION_OFFSET`, `DCT_SECTION_OFFSET`, `QUEUE_SIZE`, `PIO_SECTION_OFFSET`.
- **W1C** registers: `INTR_STATUS`, `PIO_INTR_STATUS` (latched bits only).

5.1 Threshold / interrupt level

`irq_level()` and the `PIO_INTR_STATUS` read path compute the level bits from the current queue occupancy:

```
TX_THLD      : (tx_depth - tx_q.size()) >= 2^(TX_BUF+1)
RX_THLD      : rx_q.size() >= 2^(RX_BUF+1)
CMD_QUEUE_READY: (cmd_depth - cmd_q.size()) >= CMD_EMPTY_BUF_THLD
RESP_READY    : resp_q.size() >= RESP_BUF_THLD
IBI_STATUS_THLD: ibi_q.size() >= IBI_STATUS_THLD
```

The readable status is `latched | level`, masked to the valid bit set.

6. Transaction engine (`process_command`)

1. Pop the front command; ignore if the bus is disabled.
2. Decode the model descriptor fields (§7 of the spec).
3. Resolve `dynamic_addr = dat[dev_index*2] >> 16 & 0x7F`.
4. Build an `i3c_xfer`; classify into `PrivateWrite/Read` or `CccWrite/Read`.
5. **Writes**: drain `ceil(len/4)` TX DWORDs → `write_data`; underflow ⇒ `OverflowUnder` + `TRANSFER_ERR_STAT`.
6. Invoke `bus_model` (or NACK if none). NACK ⇒ `AddressNack` + `TRANSFER_ERR_STAT`.
7. **Reads**: pack `read_data` into RX DWORDs; RX overflow ⇒ `OverflowUnder`.
8. Build and push the response descriptor; full response queue ⇒ `HC_INTERNAL_ERR_STAT`.
9. `schedule_recompute()`.

The `i3c_xfer` struct and `bus_model_fn` callback form the test/integration seam: a bench (or a higher-level bus model) emulates the wire.

7. IBI injection (`inject_ibi`)

Builds the IBI status descriptor (`addr<<1 | len<<8`), then appends the payload DWORDs (little-endian, zero-padded). Fails (returns `false`) if the IBI queue cannot hold the descriptor plus payload. Raises `IBI_STATUS_THLD_STAT` via the level computation and `schedule_recompute()`.

8. Output driving (`output_method`)

For every instance: `irq_o[i] = irq_level()`. The bus and recovery outputs are held idle (`scl_o/sda_o = 1`, `*_oe_o = 0`, `sel_od_pp_o = 0`, `recovery_* = 0`) — the documented bit-level abstraction.

9. Single-driver & determinism

- Only `output_method` writes signals.
 - All state changes funnel through `schedule_recompute()` (zero-delay) and, for transactions, `schedule_xfer()` (`xfer_delay`).
 - `inject_ibi` and `set_bus_model` are back-door APIs intended for the test bench / integration harness; they mutate state and recompute but never drive signals directly.
-

10. Build / packaging

- `CMakeLists.txt` builds `libsmc_i3c_controller.a` and the two benches.
- C++ standard is auto-detected from the linked SystemC via `smc/cmake/SmcSystemCStd.cmake` (C++20 against the repo toolchain).
- CCI is discovered via `-DCCI_HOME` / `$CCI_HOME` / default.
- `run_tests.sh` supports `Release`, `--asan`, `--coverage`, `--ctest`, `--clean`.